

PROJECT AND TEAM INFORMATION

Project Title

CompressIt — File Compression, Backup and Search Utility Engine

Student / Team Information

<i>Team Name:</i> <i>Team #</i>	
Team member 1 (Team Lead) <i>(Last Name, name: student ID: email, picture):</i>	<i>Joshi, Udhav –</i> <i>2510370490</i> <i>2510370490@geu.ac.in</i> 
Team member 2 <i>(Last Name, name: student ID: email, picture):</i>	<i>Patni, Ankush –</i> <i>2510010535</i> <i>2510010535@geu.ac.in</i> 
Team member 3 <i>(Last Name, name: student ID: email, picture):</i>	<i>Rastogi, Kanav –</i> <i>2510370429</i> <i>2510370429@geu.ac.in</i> 
Team member 4 <i>(Last Name, name: student ID: email, picture):</i>	<i>Nayak, Ishaan –</i> <i>2510360112</i> <i>2510360112@geu.ac.in</i> 

PROPOSAL DESCRIPTION

Motivation

Managing many files can become difficult because they take up a lot of storage, duplicate files use unnecessary space, and making backups repeatedly can take time. searching for specific information in multiple text file can also be hard.

Our project, CompressIt, is designed to bring these tasks into one application. In it we'll provide file compression, duplicate-file backup, detection, and text searching in a single tool. we'll use Huffman Coding for compression, hashing for detecting duplicate/changed files, and an inverted index for faster text searching.

This project will also help us to understand how the Data Structures and OOP concepts that we are learning can be applied to a real application.

State of the Art / Current solution

There are quite a few tools available for file management. for example, you can use WinRAR to compress files . you can also have different backup apps that create copies of important files . some backup systems also support incremental backups, where only the changed files are saved.

There are also different search tools that can search through multiple files, or use indexes to make searching faster.

CompressIt is all about combining these features into one application. Our project will provide all of them together instead of different tools for compression, backup, duplicate detection and searching.

Also, we want to implement the main algorithms and data structures ourselves in C++, so that we can understand their practical application.

Project Goals and Milestones

Project Goals

Phase 1 - Planning and Designing

- *Complete the requirement and features of the project.*
- *Design the system architecture and plan the application of Data Structures and OOP*

Phase 2 – Developing

- *Develop the modules in C++ and file handling. the functionalities of Huffman compression, hashing, incremental backup, and inverted-index based searching.*

Phase 3 – Integrating and Testing

- *Integrate all the modules to form a utility.*
- *Test the whole system and assess the results like compression ratio, storage space saved, and search efficiency.*

Project Milestones

1. *Define the specifications, features, and the overall architecture of the system.*
2. *Implement the basics of file handling and management in C++.*
3. *Implement Huffman tree and compression/decompression operations.*
4. *Implement hashing operations for duplicate and modified file identification and incremental backup.*
5. *Build the inverted index and text search module.*
6. *Combine the modules and ensure that CompressIt is functioning properly with different types of files.*

Project Approach

We will design CompressIt incrementally in C++. at first, we will figure out the overall design and architecture of the program and split it into various modules like compression, backup, duplicate detection, and text search.

To compress the files, we will employ Huffman Coding algorithm with priority queue and binary tree. for comparison and detection of duplicates or modified files, we will use hashing. Backup feature of the system will make only changes to the files rather than backing up all files.

For searching, we will use hashing to build an inverted index for fast word search among files.

C++ will be the core language used in development of the program. We will use STL, file handling, and OOP principles during programming. To maintain proper structure of the code, we will try to use classes like Compressor, BackupManager, FileManager, and SearchEngine.

Initially, we will develop each module separately and then unite them into one application for testing and analysis. We will evaluate the compression ratio, saved storage space from backup and the speed of searching.

DATA STRUCTURE USED:

Binary Tree: Used in huffman compression where characters are stored in a Huffman tree, paths from the root to characters makes codes.

Priority Queue: Used in huffman compression where the two frequent nodes are selected continuously to make the Huffman tree.

Hash Table: Used in duplicate detection where file hashes are stored and compared for duplicate file detection.

Hashing: Used in backup where the present file hash is compared with the past hash for change detection.

Inverted Index: Used in text search where each word is mapped to the files and locations where it occurs.

Vector: Used to store information about files, frequencies of characters and search results which can grow dynamically.

Sorting: Used to arrange search results or file information on basis of frequencies, size or name.

Stack: Used for history versioning where the backup version comes first for restoration purpose.

OOP CONCEPTS USED:

Classes and Objects: Used to modularize code like Compressor, BackupManager, FileManager and SearchEngine.

Encapsulation: Used to keep the data and functions inside the class of a particular module.

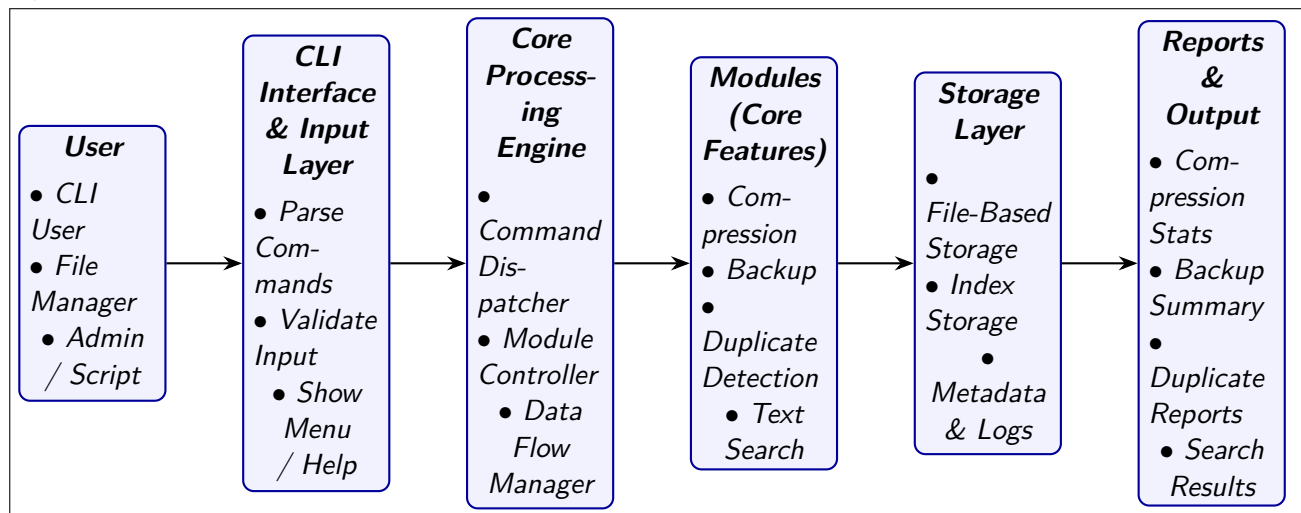
Abstraction: Used to hide the process of compression, backup, and search modules.

Constructors: Used to initialize objects and their required data.

STL: Used for C++ data structures such as priority_queue and hash-based containers.

File Handling: Used to read, write, compress, and restore files.

System Architecture



Project Outcome / Deliverables

The primary objective of the current project is the development of a C++ file management utility named as *CompressIt*. This utility will integrate various file management functions into a single utility application including compression and decompression of files using Huffman Coding, incremental backups, duplicate file detection through hashing, and searching for text across multiple files using inverted index. The application will contain a user-friendly command-line interface that allows users to take advantage of its various capabilities. In addition to this, after completing the development phase, testing of this application will be done based on some performance parameters including the compression ratio, savings due to backup, and search speed.

Assumptions

It is assumed that the user will deal mostly with standard file formats like text files, code files, log files, and documents.

The software will be running on the local machine and will need sufficient memory to make backups. It is assumed that the files are accessible, readable, and not password protected or corrupted.

For the first version, the software will concentrate on compression, backup, duplications check and text search.

References

1. H. Jeong, G. Seo, and E. Hwang, "Lossless Data Compression with Bit-back Coding on Massive Smart Meter Data," *2022 IEEE International Conference on Big Data (Big Data)*, 2022, pp. 6667–6669.
2. C. Rivera, S. Di, J. Tian, X. Yu, D. Tao, and F. Cappello, "Optimizing Huffman Decoding for Error-Bounded Lossy Compression on GPUs," *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 717–727, 2022.
3. A. Jackowski et al., "ObjDedup: High-Throughput Object Storage Layer for Backup Systems With Block-Level Deduplication," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 7, pp. 2180–2197, 2023.
4. C. Guo, W. Li, X. Tang, K.-K. R. Choo, and Y. Liu, "Forward Private Verifiable Dynamic Searchable Symmetric Encryption With Efficient Conjunctive Query," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 2, pp. 746–763, 2024.